

CCDSALG Term 3, AY 2025 – 2026
MCO1 DOCUMENTATION – Convex Hull (Application of Stack Data Structure & Sorting Algorithms)

I. DECLARATION OF INTELLECTUAL HONESTY & ETHICAL USE OF GENERATIVE AI

1. We declare that the project we are submitting is the product of our own work augmented with responsible and ethical use of generative AI as encoded in Table 1 and Table 2 below.
2. We declare that all AI-assisted contributions have been critically examined by our group. We acknowledge the limitations of Generative AI systems, including possible errors or biases, and accept full responsibility and accountability for all contents in all our project deliverables.
3. We also declare that each member COOPERATED and contributed EQUALLY to the project as detailed in Table 3 below.
4. No part of our work was shared with another person outside of our group.

// Instruction #1: Encode your names, and affix your e-signature to attest to the declaration above.

CHENG, DERK SIMONE C. S04



CORTEZ, PAUL HAVEN C. S09



SORILLA, JOSHUA DEL F. S09



// Instruction #2: Fill-up the column 2 of Table 1 below.

For each of the following tasks, identify the degree of AI use among the following:

- **NONE** - No AI was used for this task
- **SCAFFOLDING** - AI was used to build the foundation for this task, e.g., brainstorming, solution ideation
- **REFINEMENT** - AI was used to review, provide feedback, and revise outputs for this task
- **GENERATION** - AI was used to generate portions of the output for this task

Table 1. Responsible and Ethical Use of AI Details

TASKS	DEGREE OF AI USE & BRIEF EXPLANATION
T1: Understand the project specifications (problem) & assign responsibilities.	SCAFFOLDING - we used AI to get a more detailed explanation about the convex hull problem.
T2: Design, implement and test your stack data structure module.	SCAFFOLDING - we used AI to get a more detailed explanation to understand and make a checklist for reference
T3: Implement and test your TWO sorting algorithms module.	none - bubble sort Scaffolding - Quick Sort
T4: Implement two versions of Graham's Scan algorithm.	REFINEMENT: used AI to get feedback on how we can improve and then we refined the code based on the feedback.

T5: Implement two driver programs to test the two Graham's Scan algorithm versions.	REFINEMENT: We used AI to get a review/feedback on how we can improve the document. We refined the code based on the feedback that we deemed reasonable and logical.
T6: Black box test your Graham's Scan Algorithm implementation.	none
Task 7. Document your project.	REFINEMENT: We used AI to get a review/feedback on how we can improve the document. We refined the document based on the feedback that we deemed reasonable and logical.

// Instruction #3: Fill up Table 2. List explicitly ALL AI tools that you employed, and justify their purpose/role in your project.

Table 2. List of AI Tools Used

AI TOOL USED (embed the link)	JUSTIFICATION/PURPOSE/ROLE
Tool #1 Gemini https://gemini.google.com/app	To refine code, for double checking if syntax was correct, indexations and missing semi-colons. Some generation for logic and clarifications about the convex hull and graham scan algorithm aside from visualgo. Also used as a monitor for grammar checks and structure and violation monitoring for the documentation.

//... add more lines as you deem necessary

// Instruction #4: Fill up columns 1 and 2 of Table 3.

Table 3. Individual Contributions to the Project

NAME	CONTRIBUTION DETAILS
CHENG, DERK SIMONE C.	T1: Lead programmer for sort.c and sort.h T2: Tester for multiple modules T3: Created the Inputs 1-10 T4: Support in documentation T5: Lead the Git in putting testers and pushing code T6: Lead programmer for test_fast.c T7:
CORTEZ, PAUL HAVEN C.	T1: Led discussion about the Convex Hull and how it worked T2: Lead programmer for stack.c and stack.h T3: Tester for multiple modules

	T4: Support in documentation T5: Lead programmer for graham_fast.c and graham_slow.c T6:
SORILLA, JOSHUA D.	T1: Tester for multiple modules T2: Next-to-Top Function and test. function contributor T3: Header Fixes/syntax/parameter checks T4: Bug fixes on 'top' function parameter T5: Lead for documentation T6: Lead programmer for test_sort.c and test_stack.c T7: Lead programmer for test_slow.c

II. SUBMISSION CHECKLIST

// Instruction #5: Fill up Column 3 of Table 4 below.

Put a check mark as specified in the 3rd column of the table below. Please make sure that you use the same file names and that you encoded the appropriate file contents. You may create/add other source files (for example, for reading and writing data).

Table 4. File Submission Checklist

FILE	DESCRIPTION	Put a check mark ✓ below to indicate that you submitted a required file
stack.h	stack data structure header file	✓
stack.c	stack data structure C source file	✓
test_stack.c	driver program for the stack module	✓
sort.h	"slow" and "fast" sorting algo header file	✓
sort.c	"slow" and "fast" sorting algo C source file	✓
test_sort.c	driver program for the sorting module	✓
graham_slow.c	Graham's Scan algorithm slow version (using the "slow" sorting algorithm)	✓
graham_fast.c	Graham's Scan algorithm fast version (using the "fast" sorting algorithm)	✓
test_slow.c	drive program for the "slow" version	✓
test_fast.c	drive program for the for the "fast" version	✓
INPUT1.TXT to INPUT10.TXT	10 sample input files (with increasing values of n)	✓
OUTPUT1.TXT to OUTPUT10.TXT	10 sample corresponding output files	✓
GROUPNUMBER.PDF	the PDF file of this document	✓

III. INSTRUCTIONS ON HOW TO COMPILE & RUN THE PROGRAMS

// Instruction #6: Indicate how to compile your source files, and how to RUN your exe files from the COMMAND LINE.

Examples are shown below highlighted in yellow. Replace them accordingly. Make sure that all your group members test what you typed below because I will follow them verbatim (copy/paste as is). I will initially test your solution using a sample input text file that you submitted. Thereafter, I will run it again using my own test data:

- How to compile from the command line
C:\MCO> gcc -Wall test_slow.c -o slow.exe
C:\MCO> gcc -Wall test_fast.c -o fast.exe
- How to run from command line
C:\MCO>slow
C:\MCO>fast

Next, answer the following questions:

- Is there a compilation (syntax error) in your codes? (YES or NO). NO
WARNING: the project will automatically be graded with a score of 0 if there is syntax error in any of the submitted source code files. Please make sure that your submission does not have a syntax error.
- Is there any compilation warning in your codes? (YES or NO) NO
WARNING: there will be a 1 point deduction for every unique compiler warning. Please make sure that your submission does not have a compiler warning.

IV. DISCLOSURE OF ERROR(S)

// Instruction #7: Disclose IN DETAIL what is/are NOT working correctly in your solution.

Please be honest about this. NON-DISCLOSURE will result in severe point deduction! Explain briefly the reason why your group was not able to make it work.

The following are NOT working (buggy):

- NONE - The program successfully compiles without errors or warnings and correctly computes the convex hull for all tested inputs, including the maximum 32,768 stress test.

We were not able to make them work because:

- N/A - All required features, including the stack structure, sorting algorithms, and file I/O formatting, are fully operational.

V. TEST RESULTS

// Instruction #8: Fill up Table 5 below.

Based on the exhaustive empirical testing that you did, fill-up the Comparison Table below that shows the performance between the “slow” version versus the “fast” version. Test for 10 different values of n , starting with $n = 2^6 = 64$ points.

Table 5. Performance Comparison Between “Slow” and “Fast” Versions

Test Case #	n (input size)	“Slow” version execution time in ms	“Fast” version execution time in ms
1	$2^6 = 64$	0.092ms	0.035ms
2	2^7	0.163ms	0.069ms
3	2^8	0.625ms	0.137ms
4	2^9	2.324ms	0.323ms
5	2^{10}	9.299ms	0.594ms
6	2^{11}	27.995ms	1.087ms
7	2^{12}	106.719ms	2.323ms
8	2^{13}	440.168ms	5.393ms
9	2^{14}	1929.246.ms	6.571ms
10	$2^{15} = 32768$	7850.415ms	13.565ms

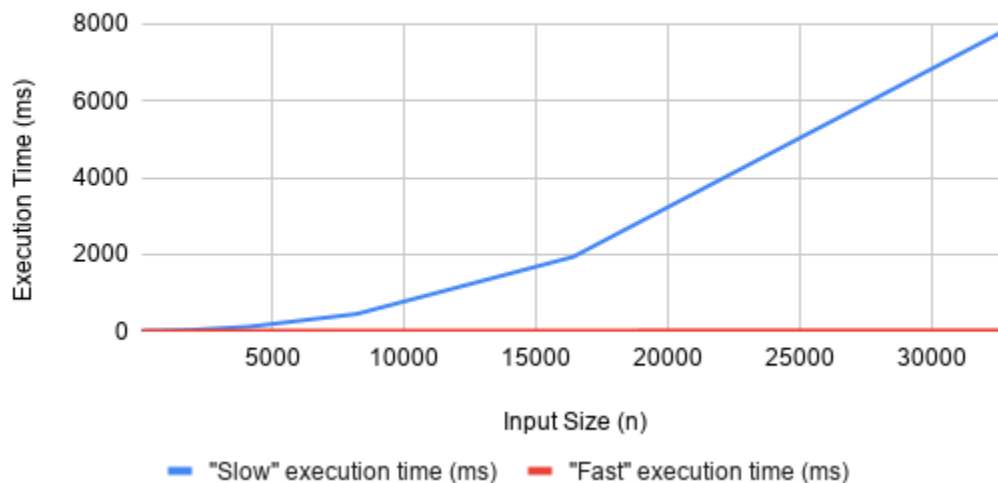
NOTE: Make sure that you fill-up the table properly. It contributes 4 out of 15 points for the Documentation.

// Instruction #9: Provide a graph showing a visual comparison of the “slow” and “fast” versions.

Create a graph (for example using Excel) based on the Comparison Table that you filled-up above. The x-axis should be the values of n and the y axis should be the execution time in milliseconds (ms).

There should be two line graphs superimposed on the same x y axes, one for the “slow” and the other for the “fast”. DO NOT SUBMIT TWO separate graphs. Replace the image below with your own.

Performance Comparison Between "Slow" and "Fast" Versions



NOTE: Make sure that you provide a graph based on your comparison table data above. It contributes 4 out of 15 points for the Documentation.

VI. ANALYSIS OF RESULTS

// Instruction #10: Compare and analyze the growth rate behaviors of the “slow” and “fast” versions based on the Comparison Table and the graphs above.

Answer the following questions:

a. What do you think is the growth rate behavior of the “slow” version?

The growth rate for the "slow" version (Bubble Sort) is $O(n^2)$ or quadratic. We can clearly see this in our graph where the blue line shoots up really fast as n gets larger. This happens because Bubble Sort uses nested loops to check and swap adjacent elements one by one. So, if we double the input size, the time it takes roughly quadruples. Our test results prove this: when we went from 16,384 points to 32,768 points, the execution time jumped from about 1,929 ms to 7,850 ms, which is almost exactly a 4x increase.

b. What do you think is the growth rate behavior of the “fast” version?

The growth rate of the "fast" version (Quick Sort) is $O(n \log n)$. Looking at our graph, the red line stays practically flat near the x-axis compared to the slow version, even at the maximum input size. This is because Quick Sort is a divide-and-conquer algorithm. Instead of checking everything repeatedly, it picks a pivot and splits the array into smaller parts to sort them recursively. Because of this, it scales really well. Even for our massive stress test of 32,768 points, the fast version finished in just 13.565 ms.

c. What do you think is/are the factor/s that make the “fast” version compute the results faster than the “slow” version?

We think the "fast" version is way faster than the "slow" version because of a few main reasons:

Divide and Conquer: Quick Sort breaks the problem down into smaller chunks, so it does significantly fewer comparisons overall. Bubble Sort wastes a lot of time doing redundant passes across the unsorted parts of the array over and over again.

More efficient swapping: Bubble Sort only swaps elements right next to each other, so an element takes a long time to travel to its correct spot. Quick Sort swaps elements across larger distances during its partition step, which moves data to where it belongs much more efficiently.

Time Complexity: $O(n \log n)$ just scales way better than $O(n^2)$. As n gets large, Bubble Sort gets overwhelmed by the massive amount of loop iterations it has to process (causing the huge 7,800+ ms spike in our graph), while Quick Sort handles the large input easily.

NOTE: Make sure that you provide cohesive answers to the three questions above. This part contributes 4 out of 15 points for the Documentation.

VII. SELF-EVALUATION

// Instruction #11: Fill-up Table 6 below.

Refer to the rubric in the project specs. It is suggested that you do an individual self-assessment first. Thereafter, compute the average evaluation for your group, and encode it below.

Table 6. Self-Evaluation

REQUIREMENT	AVE. OF SELF-ASSESSMENT
1. Stack	<u>17</u> (max. 20 points)
2. Sorting algorithms	<u>7</u> (max. 10 points)
3. Graham's Scan algorithm	<u>34</u> (max. 40 points)
4. Project Documentation and Analysis	<u>13</u> (max. 15 points)
5. Compliance with Instructions	<u>5</u> (max. 5 points)

REQUIREMENT	AVE. OF SELF-ASSESSMENT
-------------	-------------------------

TOTAL SCORE

76 over 90.

Note that the remaining 10 points (over 100) will come from the project demo.

NOTE: The evaluation that the instructor will give is not necessarily going to be the same as what you indicated above. The self-assessment is for your own reference only...

– The End –

サルバドル・フロランテ